```
Start coding or generate with AI.
```

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
```

Step 1: install pyspark and import packages

```
!pip install pyspark
```

```
Requirement already satisfied: pyspark in /usr/local/lib/python3.12/dist-packages (4.0.2)
Requirement already satisfied: py4j<0.10.9.10,>=0.10.9.7 in /usr/local/lib/python3.12/dist-packages (from pyspark) (0.10.9.9)
```

```
from pyspark.sql import SparkSession
import pyspark.sql.functions as F

from pyspark.ml import Pipeline
from pyspark.ml.feature import VectorAssembler, StandardScaler, ChiSqSelector
from pyspark.ml.classification import LogisticRegression, NaiveBayes, DecisionTreeClassifier, RandomForestClassifier
from pyspark.ml.evaluation import BinaryClassificationEvaluator, MulticlassClassificationEvaluator
```

Step 2: create spark session

```
spark = SparkSession.builder \
  .appName("Spambase_HW3") \
  .getOrCreate()
```

Step 3: define file path

```
data_path = "/content/drive/MyDrive/Colab Notebooks/spambase/spambase.data"

# to grader: note that you may need to update the path
```

Step 4: define column names

```
column_names = [
    "word_freq_make",
    "word_freq_address",
    "word_freq_all",
    "word_freq_3d",
    "word_freq_our",
    "word_freq_over",
    "word_freq_remove",
    "word_freq_internet",
    "word_freq_order",
    "word_freq_mail",
    "word_freq_receive",
    "word_freq_will",
    "word_freq_people",
    "word_freq_report",
    "word_freq_addresses",
    "word_freq_free",
    "word_freq_business",
    "word_freq_email",
    "word_freq_you",
    "word_freq_credit",
    "word_freq_your",
    "word_freq_font",
    "word_freq_000",
    "word_freq_money",
    "word_freq_hp",
    "word_freq_hpl",
    "word_freq_george",
    "word_freq_650",
    "word_freq_lab",
    "word_freq_labs",
    "word_freq_telnet",
    "word_freq_857",
    "word_freq_data",
    "word_freq_415",
    "word_freq_85",
    "word_freq_technology",
    "word_freq_1999",
    "word_freq_parts",
    "word_freq_pm",
    "word_freq_direct",
    "word_freq_cs",
    "word_freq_meeting",
    "word_freq_original",
    "word_freq_project",
    "word_freq_re",
```

```
    "word_freq_edu",
    "word_freq_table",
    "word_freq_conference",
    "char_freq_semicolon",
    "char_freq_left_paren",
    "char_freq_left_bracket",
    "char_freq_exclamation",
    "char_freq_dollar",
    "char_freq_hash",
    "capital_run_length_average",
    "capital_run_length_longest",
    "capital_run_length_total",
    "spam"
]
```

Step 5: read the data

```
df = spark.read \
  .option("header", False) \
  .option("inferSchema", True) \
  .csv(data_path)

df = df.toDF(*column_names)
```

Step 6: inspect

```
df.printSchema()
```

```
 |-- word_freq_address: double (nullable = true)
 |-- word_freq_all: double (nullable = true)
 |-- word_freq_3d: double (nullable = true)
 |-- word_freq_our: double (nullable = true)
 |-- word_freq_over: double (nullable = true)
 |-- word_freq_remove: double (nullable = true)
 |-- word_freq_internet: double (nullable = true)
 |-- word_freq_order: double (nullable = true)
 |-- word_freq_mail: double (nullable = true)
 |-- word_freq_receive: double (nullable = true)
 |-- word_freq_will: double (nullable = true)
 |-- word_freq_people: double (nullable = true)
 |-- word_freq_report: double (nullable = true)
 |-- word_freq_addresses: double (nullable = true)
```

```
 |-- word_freq_email: double (nullable = true)
 |-- word_freq_you: double (nullable = true)
 |-- word_freq_credit: double (nullable = true)
 |-- word_freq_your: double (nullable = true)
 |-- word_freq_font: double (nullable = true)
 |-- word_freq_000: double (nullable = true)
 |-- word_freq_money: double (nullable = true)
 |-- word_freq_hp: double (nullable = true)
 |-- word_freq_hpl: double (nullable = true)
 |-- word_freq_george: double (nullable = true)
 |-- word_freq_650: double (nullable = true)
 |-- word_freq_lab: double (nullable = true)
 |-- word_freq_labs: double (nullable = true)
 |-- word_freq_telnet: double (nullable = true)
 |-- word_freq_857: double (nullable = true)
 |-- word_freq_data: double (nullable = true)
 |-- word_freq_415: double (nullable = true)
 |-- word_freq_85: double (nullable = true)
 |-- word_freq_technology: double (nullable = true)
 |-- word_freq_1999: double (nullable = true)
 |-- word_freq_parts: double (nullable = true)
 |-- word_freq_pm: double (nullable = true)
 |-- word_freq_direct: double (nullable = true)
 |-- word_freq_cs: double (nullable = true)
 |-- word_freq_meeting: double (nullable = true)
 |-- word_freq_original: double (nullable = true)
 |-- word_freq_project: double (nullable = true)
 |-- word_freq_re: double (nullable = true)
 |-- word_freq_edu: double (nullable = true)
 |-- word_freq_table: double (nullable = true)
 |-- word_freq_conference: double (nullable = true)
 |-- char_freq_semicolon: double (nullable = true)
 |-- char_freq_left_paren: double (nullable = true)
 |-- char_freq_left_bracket: double (nullable = true)
 |-- char_freq_exclamation: double (nullable = true)
 |-- char_freq_dollar: double (nullable = true)
 |-- char_freq_hash: double (nullable = true)
 |-- capital_run_length_average: double (nullable = true)
 |-- capital_run_length_longest: integer (nullable = true)
 |-- capital_run_length_total: integer (nullable = true)
 |-- spam: integer (nullable = true)
```

```
df.show(10, truncate=False)
```

```
+--------------+-----------------+-------------+------------+-------------+--------------+----------------+------------------+---------
|word_freq_make|word_freq_address|word_freq_all|word_freq_3d|word_freq_our|word_freq_over|word_freq_remove|word_freq_internet|word_freq
+--------------+-----------------+-------------+------------+-------------+--------------+----------------+------------------+---------
|0.0           |0.64             |0.64         |0.0         |0.32         |0.0           |0.0             |0.0               |0.0
|0.21          |0.28             |0.5          |0.0         |0.14         |0.28          |0.21            |0.07              |0.0
|0.06          |0.0              |0.71         |0.0         |1.23         |0.19          |0.19            |0.12              |0.64
```

```
|0.0          |0.0             |0.0         |0.0         |0.63        |0.0          |0.31           |0.63             |0.31
|0.0          |0.0             |0.0         |0.0         |0.63        |0.0          |0.31           |0.63             |0.31
|0.0          |0.0             |0.0         |0.0         |1.85        |0.0          |0.0            |1.85             |0.0
|0.0          |0.0             |0.0         |0.0         |1.92        |0.0          |0.0            |0.0              |0.0
|0.0          |0.0             |0.0         |0.0         |1.88        |0.0          |0.0            |1.88             |0.0
|0.15         |0.0             |0.46        |0.0         |0.61        |0.0          |0.3            |0.0              |0.92
|0.06         |0.12            |0.77        |0.0         |0.19        |0.32         |0.38           |0.0              |0.06
+-------------+----------------+------------+------------+------------+-------------+---------------+-----------------+---------
only showing top 10 rows
```

```
print("row count:", df.count())

print("column count:", len(df.columns))
```

```
row count: 4601
column count: 58
```

```
total_rows = df.count()

print("missing values for every column:\n")

for col in df.columns:

  missing_count = df.filter(F.col(col).isNull()).count()
  missing_pct = (missing_count / total_rows) * 100

  print(f"{col}")
  print(f"  missing values: {missing_count}")
  print(f"  percentage missing: {missing_pct:.2f}%\n")
```

```
char_freq_semicolon
  missing values: 0
  percentage missing: 0.00%

char_freq_left_paren
  missing values: 0
  percentage missing: 0.00%

char_freq_left_bracket
  missing values: 0
  percentage missing: 0.00%

char_freq_exclamation
  missing values: 0
  percentage missing: 0.00%

char_freq_dollar
  missing values: 0
  percentage missing: 0.00%

char_freq_hash
  missing values: 0
  percentage missing: 0.00%

capital_run_length_average
  missing values: 0
  percentage missing: 0.00%

capital_run_length_longest
  missing values: 0
  percentage missing: 0.00%

capital_run_length_total
  missing values: 0
  percentage missing: 0.00%

spam
  missing values: 0
  percentage missing: 0.00%
```

```
# summary statistics for all variables
df.describe().show(truncate=False)
```

```
+-------------------+-------------------+------------------+-------------------+--------------------+-------------------+---------------
|word_freq_mail     |word_freq_receive  |word_freq_will    |word_freq_people   |word_freq_report    |word_freq_addresses|word_freq_free
+-------------------+-------------------+------------------+-------------------+--------------------+-------------------+---------------
|4601               |4601               |4601              |4601               |4601                |4601               |4601
|0.23941317104977197|0.05982395131493154|0.5417018039556623|0.09392958052597254|0.058626385568354686|0.04920452075635739|0.2488480765051
```

```
|0.6447553994517121 |0.20154466405444352|0.8616984712807684|0.3010358035849219 |0.3351838297557584  |0.2588434513226129 |0.8257917011289
|0.0                |0.0                |0.0               |0.0                |0.0                 |0.0                |0.0
|18.18              |2.61               |9.67              |5.55               |10.0                |4.41               |20.0
+-------------------+-------------------+------------------+-------------------+--------------------+-------------------+---------------
```

The dataset contains 4601 email records and 58 columns: 57 predictors and 1 class label. All predictors are numeric and there are no missing values, so the data required very little cleaning before modelling. The class split is about 60.6% non-spam and 39.4% spam, so the classes are somewhat imbalanced but not extremely so.

No missing columns, yay!

explore y variable

```
df.groupBy("spam").count().orderBy("spam").show()
```

```
+----+-----+
|spam|count|
+----+-----+
|   0| 2788|
|   1| 1813|
+----+-----+
```

```
df.groupBy("spam").count() \
  .withColumn("percentage", F.col("count") / total_rows * 100) \
  .orderBy("spam").show()
```

```
+----+-----+------------------+
|spam|count|        percentage|
+----+-----+------------------+
|   0| 2788| 60.59552271245382|
|   1| 1813|39.404477287546186|
+----+-----+------------------+
```

Roughly a 60-40 split between ham and spam

Step 7: cast cols to double

```
for c in df.columns:

  df = df.withColumn(c, F.col(c).cast("double"))
```

Step 8: define label and feature cols

```
label_col = "spam"
feature_cols = [c for c in df.columns if c != label_col]

print("num of feature cols:", len(feature_cols))
```

```
num of feature cols: 57
```

I kept all 57 predictors as features and used spam as the target variable. Since the dataset already contains engineered numeric features that are directly relevant to spam detection, there was no need to drop columns at this stage.

Step 9: split into train and test sets

```
train_df, test_df = df.randomSplit([0.7, 0.3], seed=1)

print("training set size:", train_df.count())
print("test set size:", test_df.count())
print("training proportion:", train_df.count() / total_rows)
print("test proportion:", test_df.count() / total_rows)
```

```
training set size: 3212
test set size: 1389
training proportion: 0.6981091067159313
test proportion: 0.3018908932840687
```

I split the data into training and test sets using a 70/30 split. This lets you train the models on one portion of the data and evaluate them on unseen emails, which gives a more realistic measure of performance.

Good to go! Now we can set up the model

Step 10: create preprocessing stages

```
#take all predictor columns and pack into 1 vector col to make one single features input
assembler = VectorAssembler(
    inputCols=feature_cols,
    outputCol="features_raw"
)

#create scaled version of the feature vector so variables on bigger scales do not dominate just because their numbers are larger
scaler = StandardScaler(
    inputCol="features_raw",
    outputCol="features_scaled",
    withStd=True,
    withMean=False
)

#run chi-square feature selection on the raw features and keep the top 20 most relevant
selector_raw = ChiSqSelector(
    numTopFeatures=20,
    featuresCol="features_raw",
    outputCol="features_selected",
    labelCol=label_col
)

#same feature selection after scaling as well to compare setups
selector_scaled = ChiSqSelector(
    numTopFeatures=20,
    featuresCol="features_scaled",
    outputCol="features_selected",
    labelCol=label_col
)
```

I set up three preprocessing components for later experiments. The assembler combines all predictors into a single feature vector, the scaler lets me test whether normalisation helps, and the chi-square selector lets me test whether a smaller subset of features performs better than the full set.

Step 11: logistic regression with raw features

I started with logistic regression as a baseline model. It is simple, and gives probability outputs that are useful later for cost-sensitive threshold tuning.

```
#logistic regression model
lr_raw = LogisticRegression(
  featuresCol="features_raw",
```

```
  labelCol="spam",
  maxIter=100)

#pipeline
pipeline_lr_raw = Pipeline(stages=[
  assembler,
  lr_raw])

#fit model
lr_raw_model = pipeline_lr_raw.fit(train_df)

#make preds
lr_raw_preds = lr_raw_model.transform(test_df)

lr_raw_preds.select("spam", "prediction", "probability").show(100, truncate=False)
```

```
|0.0 |0.0       |[0.9995517774504341,4.482225495658909E-4]  |
|1.0 |1.0       |[0.4275874960271563,0.5724125039728437]    |
|0.0 |0.0       |[0.8276478702085543,0.17235212979144565]   |
|0.0 |0.0       |[0.9924434931766465,0.007556506823353515]  |
|0.0 |0.0       |[0.969190704114049,0.03080929588595105]    |
|0.0 |0.0       |[0.9667501528949369,0.03324984710506307]   |
|0.0 |0.0       |[0.967238652136005,0.03276134786399498]    |
|0.0 |0.0       |[0.9999982675150768,1.7324849231892259E-6] |
|0.0 |0.0       |[0.8820279207846612,0.11797207921533881]   |
|0.0 |0.0       |[0.9999699494805084,3.0050519491564565E-5] |
|0.0 |0.0       |[0.9999999999999951,4.884981308350689E-15] |
|0.0 |0.0       |[1.0,0.0]                                  |
|0.0 |0.0       |[0.9999999999999465,5.3512749786932545E-14]|
|1.0 |1.0       |[0.14889965395544782,0.8511003460445522]   |
|0.0 |0.0       |[0.9017320096521082,0.0982679903478918]    |
|0.0 |0.0       |[0.9691854635357755,0.030814536464224496]  |
|0.0 |0.0       |[0.984841667134216,0.015158332865784052]   |
|0.0 |0.0       |[0.9841000919655175,0.01589990803448249]   |
|0.0 |0.0       |[0.9999981504196483,1.8495803516538345E-6] |
|0.0 |0.0       |[0.9999981504196483,1.8495803516538345E-6] |
|0.0 |0.0       |[0.9999711984747413,2.880152525874813E-5]  |
|0.0 |0.0       |[0.9999990454012138,9.545987862402683E-7]  |
|0.0 |0.0       |[0.9999997158217807,2.841782192897213E-7]  |
+----+----------+-------------------------------------------+
only showing top 100 rows
```

Step 12: evaluate logistic regression

```
#accuracy
accuracy_eval = MulticlassClassificationEvaluator(
  labelCol="spam",
  predictionCol="prediction",
  metricName="accuracy")

lr_raw_accuracy = accuracy_eval.evaluate(lr_raw_preds)
print("logistic regression accuracy:", lr_raw_accuracy)
```

```
logistic regression accuracy: 0.9244060475161987
```

```
#auc
auc_eval = BinaryClassificationEvaluator(
  labelCol="spam",
  rawPredictionCol="rawPrediction",
  metricName="areaUnderROC")

lr_raw_auc = auc_eval.evaluate(lr_raw_preds)
print("logistic regression auc:", lr_raw_auc)
```

```
logistic regression auc: 0.9741831277965731
```

```
# confusion matrix
lr_raw_preds.groupBy("spam", "prediction").count().orderBy("spam", "prediction").show()
```

```
+----+----------+-----+
|spam|prediction|count|
+----+----------+-----+
| 0.0|       0.0|  806|
| 0.0|       1.0|   44|
| 1.0|       0.0|   61|
| 1.0|       1.0|  478|
+----+----------+-----+
```

Logistic regression performed well, reaching over 92% accuracy and a strong ROC AUC. This made it a solid baseline, although there was still room to improve by trying more flexible models.

Step 13: logistic regression with scaled features

```
# logistic regression but this time on the scaled features
lr_scaled = LogisticRegression(
  featuresCol="features_scaled",
  labelCol="spam",
  maxIter=100)

pipeline_lr_scaled = Pipeline(stages=[
  assembler,
  scaler,
  lr_scaled])

lr_scaled_model = pipeline_lr_scaled.fit(train_df)
lr_scaled_preds = lr_scaled_model.transform(test_df)
lr_scaled_preds.select("spam", "prediction", "probability").show(100, truncate=False)
```

```
|0.0 |0.0       |[0.9999999998663549,1.3364509499069754E-10]|
|0.0 |0.0       |[1.0,0.0]                                  |
|0.0 |0.0       |[0.9714221114635702,0.02857788853642984]   |
```

```
|0.0 |0.0       |[0.9841000918152282,0.01589990818477182]  |
|0.0 |0.0       |[0.9999981504195922,1.8495804078311195E-6] |
|0.0 |0.0       |[0.9999981504195922,1.8495804078311195E-6] |
|0.0 |0.0       |[0.9999711984738353,2.8801526164690117E-5] |
|0.0 |0.0       |[0.9999990454011902,9.545988097769964E-7]  |
|0.0 |0.0       |[0.9999997158217694,2.8417823061399616E-7] |
+----+----------+-------------------------------------------+
only showing top 100 rows
```

```
# evaluate scaled log reg
lr_scaled_accuracy = accuracy_eval.evaluate(lr_scaled_preds)

lr_scaled_auc = auc_eval.evaluate(lr_scaled_preds)

print("scaled logistic regression accuracy:", lr_scaled_accuracy)

print("scaled logistic regression auc:", lr_scaled_auc)

lr_scaled_preds.groupBy("spam", "prediction").count().orderBy("spam", "prediction").show()
```

```
scaled logistic regression accuracy: 0.9244060475161987
scaled logistic regression auc: 0.9741831277965731
+----+----------+-----+
|spam|prediction|count|
+----+----------+-----+
| 0.0|       0.0|  806|
| 0.0|       1.0|   44|
| 1.0|       0.0|   61|
| 1.0|       1.0|  478|
+----+----------+-----+
```

Scaling did not change logistic regression performance here. The raw and scaled versions produced the same accuracy and AUC, so normalisation did not provide a practical benefit for this dataset in this setup.

Step 14: naive bayes with raw features

```
# naive bayes model
nb_raw = NaiveBayes(
    featuresCol="features_raw",
    labelCol="spam",
    modelType="multinomial")

pipeline_nb_raw = Pipeline(stages=[
    assembler,
    nb_raw])
```

```
    nb_raw])

nb_raw_model = pipeline_nb_raw.fit(train_df)

nb_raw_preds = nb_raw_model.transform(test_df)

nb_raw_preds.select("spam", "prediction", "probability").show(100, truncate=False)
```

```
|0.0 |0.0       |[0.999999999975647,2.435504777277022E-11]  |
|0.0 |0.0       |[1.0,4.3548637972833254E-36]               |
|0.0 |0.0       |[1.0,2.355597851655108E-67]                |
|0.0 |0.0       |[1.0,5.477172843424701E-37]                |
|1.0 |0.0       |[0.999999997394238,2.6057619941835353E-9]  |
|0.0 |0.0       |[1.0,3.1247944281897435E-21]               |
|0.0 |0.0       |[0.9999999998296445,1.7035552586767575E-10]|
|0.0 |0.0       |[0.9999998845684557,1.1543154436512315E-7] |
|0.0 |0.0       |[0.9989453508625009,0.0010546491374991143] |
|0.0 |0.0       |[0.9999999996719404,3.2805951036330666E-10]|
|0.0 |0.0       |[0.9999999996719404,3.2805951036330666E-10]|
|0.0 |1.0       |[0.11037412441812065,0.8896258755818794]   |
|0.0 |0.0       |[0.9999999999427112,5.728889578433677E-11] |
|0.0 |0.0       |[0.9999999596182071,4.038179304182439E-8]  |
+----+----------+-------------------------------------------+
only showing top 100 rows
```

```
#evaluate naive bayes
nb_raw_accuracy = accuracy_eval.evaluate(nb_raw_preds)

nb_raw_auc = auc_eval.evaluate(nb_raw_preds)

print("naive bayes accuracy:", nb_raw_accuracy)

print("naive bayes auc:", nb_raw_auc)

nb_raw_preds.groupBy("spam", "prediction").count().orderBy("spam", "prediction").show()
```

```
naive bayes accuracy: 0.7652987760979122
naive bayes auc: 0.3002924806286148
+----+----------+-----+
|spam|prediction|count|
+----+----------+-----+
| 0.0|       0.0|  701|
| 0.0|       1.0|  149|
| 1.0|       0.0|  177|
| 1.0|       1.0|  362|
+----+----------+-----+
```

Naive Bayes performed much worse than the other models. Its accuracy was lower, and its ROC AUC was especially poor, so it was not competitive as a final model for this task.

Step 15: decision tree

```
# decision tree model
dt = DecisionTreeClassifier(
```

```
dt = DecisionTreeClassifier(
    featuresCol="features_raw",
    labelCol="spam",
    maxDepth=5,
    seed=1)

pipeline_dt = Pipeline(stages=[
    assembler,
    dt])

dt_model = pipeline_dt.fit(train_df)

dt_preds = dt_model.transform(test_df)

dt_preds.select("spam", "prediction", "probability").show(100, truncate=False)
```

```
|0.0 |0.0       |[0.9369747899159664,0.06302521008403361] |
|1.0 |1.0       |[0.039780521262002745,0.9602194787379973]|
|0.0 |0.0       |[0.9369747899159664,0.06302521008403361] |
|0.0 |0.0       |[0.9369747899159664,0.06302521008403361] |
|0.0 |0.0       |[0.9369747899159664,0.06302521008403361] |
|0.0 |0.0       |[0.9369747899159664,0.06302521008403361] |
|0.0 |0.0       |[0.9369747899159664,0.06302521008403361] |
|0.0 |0.0       |[0.9369747899159664,0.06302521008403361] |
|0.0 |0.0       |[0.9369747899159664,0.06302521008403361] |
|0.0 |0.0       |[0.9369747899159664,0.06302521008403361] |
|0.0 |0.0       |[0.9369747899159664,0.06302521008403361] |
|0.0 |0.0       |[0.7614678899082569,0.23853211009174313] |
|0.0 |0.0       |[0.9369747899159664,0.06302521008403361] |
|1.0 |0.0       |[0.9369747899159664,0.06302521008403361] |
|0.0 |0.0       |[0.9369747899159664,0.06302521008403361] |
|0.0 |0.0       |[0.9369747899159664,0.06302521008403361] |
|0.0 |0.0       |[0.9369747899159664,0.06302521008403361] |
|0.0 |0.0       |[0.9369747899159664,0.06302521008403361] |
|0.0 |0.0       |[0.9369747899159664,0.06302521008403361] |
|0.0 |0.0       |[0.9369747899159664,0.06302521008403361] |
|0.0 |0.0       |[0.9369747899159664,0.06302521008403361] |
|0.0 |0.0       |[0.9369747899159664,0.06302521008403361] |
|0.0 |0.0       |[0.9369747899159664,0.06302521008403361] |
+----+----------+----------------------------------------+
only showing top 100 rows
```

```
# evaluate decision tree
dt_accuracy = accuracy_eval.evaluate(dt_preds)

dt_auc = auc_eval.evaluate(dt_preds)

print("decision tree accuracy:", dt_accuracy)

print("decision tree auc:", dt_auc)

dt_preds.groupBy("spam", "prediction").count().orderBy("spam", "prediction").show()
```

```
decision tree accuracy: 0.9143268538516919
decision tree auc: 0.8547266179198953
+----+----------+-----+
|spam|prediction|count|
+----+----------+-----+
| 0.0|       0.0|  811|
| 0.0|       1.0|   39|
| 1.0|       0.0|   80|
| 1.0|       1.0|  459|
+----+----------+-----+
```

The decision tree performed reasonably well, but it did not match logistic regression or random forest (comes later). That suggested a single tree was too limited to capture all of the useful patterns in the data.

Step 16: random forest

```
# random forest model
rf = RandomForestClassifier(
  featuresCol="features_raw",
  labelCol="spam",
  numTrees=100,
  maxDepth=10,
  seed=1
)
pipeline_rf = Pipeline(stages=[
  assembler,
  rf])

rf_model = pipeline_rf.fit(train_df)

rf_preds = rf_model.transform(test_df)

rf_preds.select("spam", "prediction", "probability").show(100, truncate=False)
```

```
|0.0 |0.0       |[0.9528079538975709,0.04719204610242906] |
|0.0 |0.0       |[0.9548014057743758,0.045198594225624264]|
|0.0 |0.0       |[0.9548260159965425,0.045173984003457616]|
|0.0 |0.0       |[0.8897377608453344,0.11026223915466567] |
|0.0 |0.0       |[0.8897377608453344,0.11026223915466567] |
|0.0 |0.0       |[0.7164225987319702,0.28357740126802966] |
|0.0 |0.0       |[0.956020041439611,0.04397995856038904]  |
|0.0 |0.0       |[0.9429822132888173,0.05701778671118272] |
|0.0 |0.0       |[0.9067468065120754,0.09325319348792456] |
|0.0 |0.0       |[0.95821912246591,0.0417808775340901]    |
|1.0 |1.0       |[0.24610294354282136,0.7538970564571786] |
|0.0 |0.0       |[0.9577904492279541,0.04220955077204597] |
|1.0 |1.0       |[0.05976180249171262,0.9402381975082874] |
|0.0 |0.0       |[0.9230761944849855,0.07692380551501458] |
|0.0 |0.0       |[0.9523257349553049,0.0476742650446951]  |
|0.0 |0.0       |[0.9388934514108987,0.061106548589101195]|
|0.0 |0.0       |[0.783183718922106,0.21681628107789397]  |
|0.0 |0.0       |[0.783183718922106,0.21681628107789397]  |
|0.0 |0.0       |[0.8788579483026966,0.12114205169730338] |
|0.0 |0.0       |[0.9383739263849357,0.06162607361506435] |
|0.0 |0.0       |[0.9552305356042907,0.04476946439570931] |
|0.0 |0.0       |[0.9542590197314449,0.04574098026855513] |
|0.0 |0.0       |[0.8644123622783854,0.13558763772161458] |
|0.0 |0.0       |[0.794578225535247,0.20542177446475296]  |
|1.0 |0.0       |[0.7034305115288586,0.2965694884711414]  |
|0.0 |0.0       |[0.9218969108310475,0.07810308916895245] |
|0.0 |0.0       |[0.9177149079069555,0.08228509209304442] |
|0.0 |0.0       |[0.9303307288904401,0.06966927110955985] |
|0.0 |0.0       |[0.9129327642243561,0.0870672357756439]  |
|0.0 |0.0       |[0.9235310459855618,0.07646895401443832] |
|0.0 |0.0       |[0.9235310459855618,0.07646895401443832] |
|0.0 |0.0       |[0.8996896674682786,0.10031033253172136] |
|0.0 |0.0       |[0.9509163414600355,0.0490836585399646]  |
|0.0 |0.0       |[0.8892097491184549,0.11079025088154515] |
+----+----------+-----------------------------------------+
only showing top 100 rows
```

```
# evaluate random forest
rf_accuracy = accuracy_eval.evaluate(rf_preds)

rf_auc = auc_eval.evaluate(rf_preds)

print("random forest accuracy:", rf_accuracy)

print("random forest auc:", rf_auc)

rf_preds.groupBy("spam", "prediction").count().orderBy("spam", "prediction").show()
```

```
random forest accuracy: 0.9496040316774658
random forest auc: 0.9858758048674009
```

```
+----+----------+-----+
|spam|prediction|count|
+----+----------+-----+
| 0.0|       0.0|  820|
| 0.0|       1.0|   30|
| 1.0|       0.0|   40|
| 1.0|       1.0|  499|
+----+----------+-----+
```

Step 17: test feature selection using logistic regression with selected features

```
# logistic regression using selected features
lr_selected = LogisticRegression(
  featuresCol="features_selected",
  labelCol="spam",
  maxIter=100)

pipeline_lr_selected = Pipeline(stages=[
  assembler,
  selector_raw,
  lr_selected])

lr_selected_model = pipeline_lr_selected.fit(train_df)
lr_selected_preds = lr_selected_model.transform(test_df)
lr_selected_accuracy = accuracy_eval.evaluate(lr_selected_preds)
lr_selected_auc = auc_eval.evaluate(lr_selected_preds)

print("logistic regression with selected features accuracy:", lr_selected_accuracy)
print("logistic regression with selected features auc:", lr_selected_auc)

lr_selected_preds.groupBy("spam", "prediction").count().orderBy("spam", "prediction").show()
```

```
logistic regression with selected features accuracy: 0.8617710583153347
logistic regression with selected features auc: 0.9282854960165858
+----+----------+-----+
|spam|prediction|count|
+----+----------+-----+
| 0.0|       0.0|  801|
| 0.0|       1.0|   49|
| 1.0|       0.0|  143|
| 1.0|       1.0|  396|
+----+----------+-----+
```

random forest + selected features

```
# random forest using selected features
rf_selected = RandomForestClassifier(
  featuresCol="features_selected",
  labelCol="spam",
  numTrees=100,
  maxDepth=10,
  seed=1)

pipeline_rf_selected = Pipeline(stages=[
  assembler,
  selector_raw,
  rf_selected])

rf_selected_model = pipeline_rf_selected.fit(train_df)

rf_selected_preds = rf_selected_model.transform(test_df)

rf_selected_accuracy = accuracy_eval.evaluate(rf_selected_preds)
rf_selected_auc = auc_eval.evaluate(rf_selected_preds)

print("random forest with selected features accuracy:", rf_selected_accuracy)
print("random forest with selected features auc:", rf_selected_auc)

rf_selected_preds.groupBy("spam", "prediction").count().orderBy("spam", "prediction").show()
```

```
random forest with selected features accuracy: 0.9114470842332614
random forest with selected features auc: 0.9487940630797769
+----+----------+-----+
|spam|prediction|count|
+----+----------+-----+
| 0.0|       0.0|  805|
| 0.0|       1.0|   45|
| 1.0|       0.0|   78|
| 1.0|       1.0|  461|
+----+----------+-----+
```

Feature selection reduced performance for both logistic regression and random forest. That suggests the full set of predictors contained useful signal, so keeping all features was better than restricting the models to only the top 20 selected variables.

Step 18: test different tree and forest settings

decision tree depths

```
# decision tree depth 3
dt_3 = DecisionTreeClassifier(
  featuresCol="features_raw",
  labelCol="spam",
  maxDepth=3,
  seed=1)

pipeline_dt_3 = Pipeline(stages=[
  assembler,
  dt_3])

dt_3_model = pipeline_dt_3.fit(train_df)
dt_3_preds = dt_3_model.transform(test_df)
dt_3_accuracy = accuracy_eval.evaluate(dt_3_preds)
dt_3_auc = auc_eval.evaluate(dt_3_preds)

print("decision tree depth 3 accuracy:", dt_3_accuracy)
print("decision tree depth 3 auc:", dt_3_auc)
```

```
decision tree depth 3 accuracy: 0.8898488120950324
decision tree depth 3 auc: 0.651336898395722
```

```
#decision tree depth 5
dt_5 = DecisionTreeClassifier(
  featuresCol="features_raw",
  labelCol="spam",
  maxDepth=5,
  seed=1)

pipeline_dt_5 = Pipeline(stages=[
  assembler,
  dt_5])

dt_5_model = pipeline_dt_5.fit(train_df)
dt_5_preds = dt_5_model.transform(test_df)
dt_5_accuracy = accuracy_eval.evaluate(dt_5_preds)
dt_5_auc = auc_eval.evaluate(dt_5_preds)

print("decision tree depth 5 accuracy:", dt_5_accuracy)
print("decision tree depth 5 auc:", dt_5_auc)
```

```
decision tree depth 5 accuracy: 0.9143268538516919
decision tree depth 5 auc: 0.8547266179198953
```

```
#decision tree depth 8
dt_8 = DecisionTreeClassifier(
  featuresCol="features_raw",
  labelCol="spam",
  maxDepth=8,
  seed=1)

pipeline_dt_8 = Pipeline(stages=[
  assembler,
  dt_8])

dt_8_model = pipeline_dt_8.fit(train_df)
dt_8_preds = dt_8_model.transform(test_df)
dt_8_accuracy = accuracy_eval.evaluate(dt_8_preds)
dt_8_auc = auc_eval.evaluate(dt_8_preds)

print("decision tree depth 8 accuracy:", dt_8_accuracy)
print("decision tree depth 8 auc:", dt_8_auc)
```

```
decision tree depth 8 accuracy: 0.9164866810655148
decision tree depth 8 auc: 0.8318050856706317
```

random forest settings

```
# random forest 50 trees depth 5
rf_50_5 = RandomForestClassifier(
  featuresCol="features_raw",
  labelCol="spam",
  numTrees=50,
  maxDepth=5,
  seed=1)
pipeline_rf_50_5 = Pipeline(stages=[
    assembler,
    rf_50_5])

rf_50_5_model = pipeline_rf_50_5.fit(train_df)
rf_50_5_preds = rf_50_5_model.transform(test_df)
rf_50_5_accuracy = accuracy_eval.evaluate(rf_50_5_preds)
rf_50_5_auc = auc_eval.evaluate(rf_50_5_preds)
```

```
print("random forest 50 trees depth 5 accuracy:", rf_50_5_accuracy)
print("random forest 50 trees depth 5 auc:", rf_50_5_auc)
```

```
random forest 50 trees depth 5 accuracy: 0.9344852411807055
random forest 50 trees depth 5 auc: 0.9789774091454708
```

```
# random forest 100 trees depth 5
rf_100_5 = RandomForestClassifier(
  featuresCol="features_raw",
  labelCol="spam",
  numTrees=100,
  maxDepth=5,
  seed=1)

pipeline_rf_100_5 = Pipeline(stages=[
    assembler,
    rf_100_5])

rf_100_5_model = pipeline_rf_100_5.fit(train_df)
rf_100_5_preds = rf_100_5_model.transform(test_df)
rf_100_5_accuracy = accuracy_eval.evaluate(rf_100_5_preds)
rf_100_5_auc = auc_eval.evaluate(rf_100_5_preds)

print("random forest 100 trees depth 5 accuracy:", rf_100_5_accuracy)
print("random forest 100 trees depth 5 auc:", rf_100_5_auc)
```

```
random forest 100 trees depth 5 accuracy: 0.9352051835853131
random forest 100 trees depth 5 auc: 0.9786958419731508
```

```
# random forest 100 trees depth 10
rf_100_10 = RandomForestClassifier(
    featuresCol="features_raw",
    labelCol="spam",
    numTrees=100,
    maxDepth=10,
    seed=1)

pipeline_rf_100_10 = Pipeline(stages=[
    assembler,
    rf_100_10])

rf_100_10_model = pipeline_rf_100_10.fit(train_df)
rf_100_10_preds = rf_100_10_model.transform(test_df)
rf_100_10_accuracy = accuracy_eval.evaluate(rf_100_10_preds)
rf_100_10_auc = auc_eval.evaluate(rf_100_10_preds)
```

```
print("random forest 100 trees depth 10 accuracy:", rf_100_10_accuracy)
print("random forest 100 trees depth 10 auc:", rf_100_10_auc)
```

```
random forest 100 trees depth 10 accuracy: 0.9496040316774658
random forest 100 trees depth 10 auc: 0.9858758048674009
```

Increasing model complexity improved performance up to a point. Among all the configurations tested, the random forest with 100 trees and depth 10 gave the strongest results, with the highest accuracy and the highest AUC.

Step 19: comparison table

```
results_rows = [
  ("logistic regression raw", lr_raw_accuracy, lr_raw_auc),
  ("logistic regression scaled", lr_scaled_accuracy, lr_scaled_auc),
  ("naive bayes raw", nb_raw_accuracy, nb_raw_auc),
  ("decision tree depth 3", dt_3_accuracy, dt_3_auc),
  ("decision tree depth 5", dt_5_accuracy, dt_5_auc),
  ("decision tree depth 8", dt_8_accuracy, dt_8_auc),
  ("random forest 50 trees depth 5", rf_50_5_accuracy, rf_50_5_auc),
  ("random forest 100 trees depth 5", rf_100_5_accuracy, rf_100_5_auc),
  ("random forest 100 trees depth 10", rf_100_10_accuracy, rf_100_10_auc),
  ("logistic regression selected", lr_selected_accuracy, lr_selected_auc),
  ("random forest selected", rf_selected_accuracy, rf_selected_auc)]

results_df = spark.createDataFrame(
  results_rows,
  ["model", "accuracy", "auc"])

results_df.orderBy(F.desc("accuracy")).show(truncate=False)
```

```
+-------------------------------+------------------+------------------+
|model                          |accuracy          |auc               |
+-------------------------------+------------------+------------------+
|random forest 100 trees depth 10|0.9496040316774658|0.9858758048674009|
|random forest 100 trees depth 5 |0.9352051835853131|0.9786958419731508|
|random forest 50 trees depth 5  |0.9344852411807055|0.9789774091454708|
|logistic regression raw         |0.9244060475161987|0.9741831277965731|
|logistic regression scaled      |0.9244060475161987|0.9741831277965731|
|decision tree depth 8           |0.9164866810655148|0.8318050856706317|
|decision tree depth 5           |0.9143268538516919|0.8547266179198953|
|random forest selected          |0.9114470842332614|0.9487940630797769|
|decision tree depth 3           |0.8898488120950324|0.651336898395722 |
|logistic regression selected    |0.8617710583153347|0.9282854960165858|
|naive bayes raw                 |0.7652987760979122|0.3002924806286148|
```

```
+-------------------------------+-----------------+-----------------+
```

```
best_accuracy_row = results_df.orderBy(F.desc("accuracy")).first()

print("best model in terms of accuracy:")
print(best_accuracy_row)
```

```
best model in terms of accuracy:
Row(model='random forest 100 trees depth 10', accuracy=0.9496040316774658, auc=0.9858758048674009)
```

Looking across all models, random forest with 100 trees at depth 10 performed best overall. It outperformed logistic regression, naive Bayes, and decision trees on both accuracy and ROC AUC, so I selected it as the final cost-unaware model.

Step 20: cost-sensitive threshold tuning

Accuracy is not the only thing that matters in spam filtering. In this problem, falsely marking a real email as spam is much more costly than letting a spam email slip through, so I used a 10:1 cost ratio to find the threshold that minimised average misclassification cost.

test multiple thresholds on scaled log reg first

```
from pyspark.ml.functions import vector_to_array

threshold_values = [0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9]

threshold_results = []

for t in threshold_values:
  scored = lr_scaled_preds.withColumn(
    "spam_probability",
    vector_to_array("probability")[1]
  ).withColumn(
    "custom_prediction",
    F.when(F.col("spam_probability") >= t, 1.0).otherwise(0.0)
  )

  fp = scored.filter((F.col("spam") == 0) & (F.col("custom_prediction") == 1)).count()
  fn = scored.filter((F.col("spam") == 1) & (F.col("custom_prediction") == 0)).count()
  correct = scored.filter(F.col("spam") == F.col("custom_prediction")).count()
  total = scored.count()

  accuracy = correct / total
```

```
  accuracy = correct / total
  avg_cost = (10 * fp + fn) / total

  threshold_results.append((t, accuracy, fp, fn, avg_cost))

threshold_df = spark.createDataFrame(
  threshold_results,
  ["threshold", "accuracy", "fp", "fn", "avg_cost"]
)

threshold_df.orderBy("avg_cost").show(truncate=False)
```

```
+---------+------------------+---+---+-------------------+
|threshold|accuracy          |fp |fn |avg_cost           |
+---------+------------------+---+---+-------------------+
|0.9      |0.8560115190784737|17 |183|0.25413966882649386|
|0.8      |0.896328293736501 |25 |119|0.265658747300216  |
|0.7      |0.912167026637869 |33 |89 |0.30165586753059753|
|0.6      |0.9222462203023758|35 |73 |0.3045356371490281 |
|0.5      |0.9244060475161987|44 |61 |0.36069114470842334|
|0.4      |0.9294456443484521|59 |39 |0.4528437724982001 |
|0.3      |0.9208063354931606|87 |23 |0.6429085673146149 |
|0.2      |0.8941684665226782|134|13 |0.9740820734341252 |
|0.1      |0.8236141108711303|243|2  |1.7508999280057596 |
+---------+------------------+---+---+-------------------+
```

Since the best threshold is 0.9, we can try some higher more granular threhsolding

```
from pyspark.ml.functions import vector_to_array

threshold_values = [0.90, 0.91, 0.92, 0.93, 0.94, 0.95, 0.96, 0.97, 0.98, 0.99]

threshold_results_high = []

for t in threshold_values:
  scored = lr_scaled_preds.withColumn(
    "spam_probability",
    vector_to_array("probability")[1]
  ).withColumn(
    "custom_prediction",
    F.when(F.col("spam_probability") >= t, 1.0).otherwise(0.0)
  )

  fp = scored.filter((F.col("spam") == 0) & (F.col("custom_prediction") == 1)).count()
  fn = scored.filter((F.col("spam") == 1) & (F.col("custom_prediction") == 0)).count()
```

```
  correct = scored.filter(F.col("spam") == F.col("custom_prediction")).count()
  total = scored.count()

  accuracy = correct / total
  avg_cost = (10 * fp + fn) / total

  threshold_results_high.append((t, accuracy, fp, fn, avg_cost))

threshold_df_high = spark.createDataFrame(
  threshold_results_high,
  ["threshold", "accuracy", "fp", "fn", "avg_cost"]
)

threshold_df_high.orderBy("avg_cost").show(truncate=False)
```

```
+---------+------------------+---+---+-------------------+
|threshold|accuracy          |fp |fn |avg_cost           |
+---------+------------------+---+---+-------------------+
|0.9      |0.8560115190784737|17 |183|0.25413966882649386|
|0.95     |0.8178545716342692|12 |241|0.2598992080633549 |
|0.91     |0.8480921526277898|17 |194|0.26205903527717783|
|0.94     |0.8272138228941684|14 |226|0.2634989200863931 |
|0.96     |0.8092152627789777|12 |253|0.2685385169186465 |
|0.92     |0.8416126709863211|17 |203|0.2685385169186465 |
|0.97     |0.8005759539236861|11 |266|0.2706983441324694 |
|0.93     |0.8308135349172067|16 |219|0.2728581713462923 |
|0.98     |0.7760979121670266|10 |301|0.2886969042476602 |
|0.99     |0.7588192944564435|8  |327|0.29301655867530596|
+---------+------------------+---+---+-------------------+
```

```
from pyspark.ml.functions import vector_to_array

threshold_values = [0.85, 0.855, 0.86, 0.865, 0.87, 0.875, 0.88, 0.885, 0.9, 0.905]

threshold_results_high = []

for t in threshold_values:
  scored = lr_scaled_preds.withColumn(
    "spam_probability",
    vector_to_array("probability")[1]
  ).withColumn(
    "custom_prediction",
    F.when(F.col("spam_probability") >= t, 1.0).otherwise(0.0)
  )

  fp = scored.filter((F.col("spam") == 0) & (F.col("custom_prediction") == 1)).count()
```

```
    fn = scored.filter((F.col("spam") == 1) & (F.col("custom_prediction") == 0)).count()
    correct = scored.filter(F.col("spam") == F.col("custom_prediction")).count()
    total = scored.count()

    accuracy = correct / total
    avg_cost = (10 * fp + fn) / total

    threshold_results_high.append((t, accuracy, fp, fn, avg_cost))

threshold_df_high = spark.createDataFrame(
  threshold_results_high,
  ["threshold", "accuracy", "fp", "fn", "avg_cost"]
)

threshold_df_high.orderBy("avg_cost").show(truncate=False)
```

```
+---------+------------------+---+---+-------------------+
|threshold|accuracy          |fp |fn |avg_cost           |
+---------+------------------+---+---+-------------------+
|0.875    |0.8675305975521959|18 |166|0.24910007199424047|
|0.88     |0.8660907127429806|18 |168|0.2505399568034557 |
|0.885    |0.8639308855291576|18 |171|0.2526997840172786 |
|0.9      |0.8560115190784737|17 |183|0.25413966882649386|
|0.87     |0.8675305975521959|19 |165|0.25557955363570917|
|0.86     |0.8732901367890569|20 |156|0.25629949604031677|
|0.905    |0.8531317494600432|17 |187|0.2570194384449244 |
|0.865    |0.8718502519798416|20 |158|0.257739380849532  |
|0.85     |0.8768898488120951|21 |150|0.2591792656587473 |
|0.855    |0.8747300215982722|21 |153|0.2613390928725702 |
+---------+------------------+---+---+-------------------+
```

```
BEST_THRESHOLD = 0.875
```

```
cost_preds = lr_scaled_preds.withColumn(
  "spam_probability",
  vector_to_array("probability")[1]
).withColumn(
  "custom_prediction",
  F.when(F.col("spam_probability") >= BEST_THRESHOLD, 1.0).otherwise(0.0))
```

A more detailed search showed that 0.875 was the best logistic-regression threshold among the values tested. This improved cost a lot compared with the default threshold, but I still needed to check whether another model could do even better.

```
cost_preds.groupBy("spam", "custom_prediction").count().orderBy("spam", "custom_prediction").show()
```

```
cost_preds.groupby("spam", "custom_prediction").count().orderBy("spam", "custom_prediction").show()
```

```
+----+-----------------+-----+
|spam|custom_prediction|count|
+----+-----------------+-----+
| 0.0|              0.0|  832|
| 0.0|              1.0|   18|
| 1.0|              0.0|  166|
| 1.0|              1.0|  373|
+----+-----------------+-----+
```

Now cost efficiency for the most accurate model:

Since random forest was already the strongest model on accuracy and AUC, I also tested cost-sensitive threshold tuning on its predicted probabilities. This was important because the best model for accuracy is not always used the same way when error costs are unequal.

```
from pyspark.ml.functions import vector_to_array

rf_threshold_values = [0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.85, 0.875, 0.9, 0.95]
rf_threshold_results = []

for t in rf_threshold_values:
  scored = rf_100_10_preds.withColumn(
    "spam_probability",
    vector_to_array("probability")[1]
  ).withColumn(
    "custom_prediction",
    F.when(F.col("spam_probability") >= t, 1.0).otherwise(0.0)
  )

  fp = scored.filter((F.col("spam") == 0) & (F.col("custom_prediction") == 1)).count()
  fn = scored.filter((F.col("spam") == 1) & (F.col("custom_prediction") == 0)).count()
  correct = scored.filter(F.col("spam") == F.col("custom_prediction")).count()
  total = scored.count()

  accuracy = correct / total
  avg_cost = (10 * fp + fn) / total

  rf_threshold_results.append((t, accuracy, fp, fn, avg_cost))

rf_threshold_df = spark.createDataFrame(
  rf_threshold_results,
  ["threshold", "accuracy", "fp", "fn", "avg_cost"]
)
```

```
rf_threshold_df.orderBy("avg_cost").show(truncate=False)
```

```
+---------+------------------+---+---+-------------------+
|threshold|accuracy          |fp |fn |avg_cost           |
+---------+------------------+---+---+-------------------+
|0.7      |0.9330453563714903|11 |82 |0.13822894168466524|
|0.8      |0.9056875449964003|7  |124|0.1396688264938805 |
|0.6      |0.9488840892728582|16 |55 |0.15478761699064075|
|0.85     |0.8732901367890569|7  |169|0.1720662347012239 |
|0.875    |0.8567314614830813|5  |194|0.17566594672426206|
|0.9      |0.8387329013678906|4  |220|0.18718502519798416|
|0.95     |0.775377969762419 |2  |310|0.23758099352051837|
|0.5      |0.9496040316774658|30 |40 |0.24478041756659466|
|0.4      |0.9481641468682506|44 |28 |0.3369330453563715 |
|0.3      |0.9395248380129589|67 |17 |0.4946004319654428 |
|0.2      |0.8977681785457163|132|10 |0.9575233981281498 |
|0.1      |0.7998560115190785|274|4  |1.9755219582433405 |
+---------+------------------+---+---+-------------------+
```

```
from pyspark.ml.functions import vector_to_array

rf_threshold_values_fine = [0.65, 0.675, 0.70, 0.725, 0.75, 0.775, 0.80]
rf_threshold_results_fine = []

for t in rf_threshold_values_fine:
  scored = rf_100_10_preds.withColumn(
    "spam_probability",
    vector_to_array("probability")[1]
  ).withColumn(
    "custom_prediction",
    F.when(F.col("spam_probability") >= t, 1.0).otherwise(0.0)
  )

  fp = scored.filter((F.col("spam") == 0) & (F.col("custom_prediction") == 1)).count()
  fn = scored.filter((F.col("spam") == 1) & (F.col("custom_prediction") == 0)).count()
  correct = scored.filter(F.col("spam") == F.col("custom_prediction")).count()
  total = scored.count()

  accuracy = correct / total
  avg_cost = (10 * fp + fn) / total

  rf_threshold_results_fine.append((t, accuracy, fp, fn, avg_cost))

rf_threshold_df_fine = spark.createDataFrame(
  rf_threshold_results_fine,
```

```
  ["threshold", "accuracy", "fp", "fn", "avg_cost"]
)

rf_threshold_df_fine.orderBy("avg_cost").show(truncate=False)
```

```
+---------+------------------+---+---+-------------------+
|threshold|accuracy          |fp |fn |avg_cost           |
+---------+------------------+---+---+-------------------+
|0.75     |0.9208063354931606|9  |101|0.1375089992800576 |
|0.7      |0.9330453563714903|11 |82 |0.13822894168466524|
|0.725    |0.9265658747300216|10 |92 |0.13822894168466524|
|0.8      |0.9056875449964003|7  |124|0.1396688264938805 |
|0.675    |0.937365010799136 |12 |75 |0.14038876889848811|
|0.775    |0.9114470842332614|8  |115|0.14038876889848811|
|0.65     |0.9431245500359972|13 |66 |0.14110871130309574|
+---------+------------------+---+---+-------------------+
```

```
rf_threshold_values_final = [
  0.735, 0.74, 0.745, 0.75, 0.755, 0.76, 0.765
]

rf_threshold_results_final = []

for t in rf_threshold_values_final:
  scored = rf_100_10_preds.withColumn(
    "spam_probability",
    vector_to_array("probability")[1]
  ).withColumn(
    "custom_prediction",
    F.when(F.col("spam_probability") >= t, 1.0).otherwise(0.0)
  )

  fp = scored.filter((F.col("spam") == 0) & (F.col("custom_prediction") == 1)).count()
  fn = scored.filter((F.col("spam") == 1) & (F.col("custom_prediction") == 0)).count()
  correct = scored.filter(F.col("spam") == F.col("custom_prediction")).count()
  total = scored.count()

  accuracy = correct / total
  avg_cost = (10 * fp + fn) / total

  rf_threshold_results_final.append((t, accuracy, fp, fn, avg_cost))

rf_threshold_df_final = spark.createDataFrame(
  rf_threshold_results_final,
  ["threshold", "accuracy", "fp", "fn", "avg_cost"]
```

```
)

rf_threshold_df_final.orderBy("avg_cost").show(truncate=False)
```

```
+---------+------------------+---+---+-------------------+
|threshold|accuracy          |fp |fn |avg_cost           |
+---------+------------------+---+---+-------------------+
|0.75     |0.9208063354931606|9  |101|0.1375089992800576 |
|0.735    |0.9244060475161987|10 |95 |0.14038876889848811|
|0.755    |0.91792656587473  |9  |105|0.14038876889848811|
|0.74     |0.9222462203023758|10 |98 |0.14254859611231102|
|0.76     |0.9150467962562995|9  |109|0.14326853851691865|
|0.745    |0.9215262778977682|10 |99 |0.14326853851691865|
|0.765    |0.9136069114470843|9  |111|0.1447084233261339 |
+---------+------------------+---+---+-------------------+
```

Create final cost-sensitive predictions

Random forest performed much better than logistic regression in the cost-sensitive setting too. The first threshold search suggested that the best region was around 0.7, so I narrowed the search around that area and found 0.75 to be the best.

```
BEST_COST_THRESHOLD = 0.75

rf_cost_preds = rf_100_10_preds.withColumn(
  "spam_probability",
  vector_to_array("probability")[1]
).withColumn(
  "custom_prediction",
  F.when(F.col("spam_probability") >= BEST_COST_THRESHOLD, 1.0).otherwise(0.0)
)
```

```
rf_cost_preds.groupBy("spam", "custom_prediction") \
  .count() \
  .orderBy("spam", "custom_prediction") \
  .show()
```

```
+----+-----------------+-----+
|spam|custom_prediction|count|
+----+-----------------+-----+
| 0.0|              0.0|  841|
| 0.0|              1.0|    9|
| 1.0|              0.0|  101|
| 1.0|              1.0|  438|
```

```
+----+----------------+-----+
```

Calculate the final average cost

```
fp = rf_cost_preds.filter((F.col("spam") == 0) & (F.col("custom_prediction") == 1)).count()
fn = rf_cost_preds.filter((F.col("spam") == 1) & (F.col("custom_prediction") == 0)).count()
total = rf_cost_preds.count()

rf_cost_accuracy = rf_cost_preds.filter(
  F.col("spam") == F.col("custom_prediction")
).count() / total

rf_avg_cost = (10 * fp + fn) / total

print("best cost-sensitive threshold:", BEST_COST_THRESHOLD)
print("false positives:", fp)
print("false negatives:", fn)
print("accuracy:", rf_cost_accuracy)
print("average cost:", rf_avg_cost)
```

```
best cost-sensitive threshold: 0.75
false positives: 9
false negatives: 101
accuracy: 0.9208063354931606
average cost: 0.1375089992800576
```

Now lets try cost efficiency for the decision tree model with depth 8

```
from pyspark.ml.functions import vector_to_array

# threshold tuning for decision tree depth 8
dt8_threshold_values = [0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.75, 0.8, 0.85, 0.9]
dt8_threshold_results = []

for t in dt8_threshold_values:
  scored = dt_8_preds.withColumn(
    "spam_probability",
    vector_to_array("probability")[1]
  ).withColumn(
    "custom_prediction",
    F.when(F.col("spam_probability") >= t, 1.0).otherwise(0.0)
  )
```

```
    fp = scored.filter((F.col("spam") == 0) & (F.col("custom_prediction") == 1)).count()
    fn = scored.filter((F.col("spam") == 1) & (F.col("custom_prediction") == 0)).count()
    correct = scored.filter(F.col("spam") == F.col("custom_prediction")).count()
    total = scored.count()

    accuracy = correct / total
    avg_cost = (10 * fp + fn) / total

    dt8_threshold_results.append((t, accuracy, fp, fn, avg_cost))

dt8_threshold_df = spark.createDataFrame(
  dt8_threshold_results,
  ["threshold", "accuracy", "fp", "fn", "avg_cost"]
)

dt8_threshold_df.orderBy("avg_cost").show(truncate=False)
```

```
+---------+------------------+---+---+-------------------+
|threshold|accuracy          |fp |fn |avg_cost           |
+---------+------------------+---+---+-------------------+
|0.85     |0.9136069114470843|33 |87 |0.3002159827213823 |
|0.9      |0.9136069114470843|33 |87 |0.3002159827213823 |
|0.8      |0.9157667386609071|34 |83 |0.3045356371490281 |
|0.75     |0.9150467962562995|38 |80 |0.33117350611951046|
|0.7      |0.9229661627069834|43 |64 |0.3556515478761699 |
|0.6      |0.9215262778977682|45 |64 |0.3700503959683225 |
|0.4      |0.9164866810655148|61 |55 |0.4787616990640749 |
|0.5      |0.9164866810655148|61 |55 |0.4787616990640749 |
|0.3      |0.9136069114470843|70 |50 |0.5399568034557235 |
|0.2      |0.9100071994240461|75 |50 |0.5759539236861051 |
|0.1      |0.8776097912167027|128|42 |0.9517638588912887 |
+---------+------------------+---+---+-------------------+
```

we can already see this is not going to better the rf model

Step 21: precision, recall, F1, confusion matrix, average cost

```
cost_precision_eval = MulticlassClassificationEvaluator(
    labelCol="spam",
    predictionCol="custom_prediction",
    metricName="weightedPrecision"
)

cost_recall_eval = MulticlassClassificationEvaluator(
```

```
    labelCol="spam",
    predictionCol="custom_prediction",
    metricName="weightedRecall"
)

cost_f1_eval = MulticlassClassificationEvaluator(
    labelCol="spam",
    predictionCol="custom_prediction",
    metricName="f1"
)

rf_cost_precision = cost_precision_eval.evaluate(rf_cost_preds)
rf_cost_recall = cost_recall_eval.evaluate(rf_cost_preds)
rf_cost_f1 = cost_f1_eval.evaluate(rf_cost_preds)

print("cost-sensitive precision:", rf_cost_precision)
print("cost-sensitive recall:", rf_cost_recall)
print("cost-sensitive f1:", rf_cost_f1)
```

```
cost-sensitive precision: 0.9265743483820572
cost-sensitive recall: 0.9208063354931606
cost-sensitive f1: 0.9191445751421892
```

final metrics for the best accuracy model

```
precision_eval = MulticlassClassificationEvaluator(
  labelCol="spam",
  predictionCol="prediction",
  metricName="weightedPrecision"
)

recall_eval = MulticlassClassificationEvaluator(
  labelCol="spam",
  predictionCol="prediction",
  metricName="weightedRecall"
)

f1_eval = MulticlassClassificationEvaluator(
  labelCol="spam",
  predictionCol="prediction",
  metricName="f1"
)

rf_best_precision = precision_eval.evaluate(rf_100_10_preds)
rf_best_recall = recall_eval.evaluate(rf_100_10_preds)
```

```
rf_best_f1 = f1_eval.evaluate(rf_100_10_preds)

print("best accuracy model precision:", rf_best_precision)
print("best accuracy model recall:", rf_best_recall)
print("best accuracy model f1:", rf_best_f1)
```

```
best accuracy model precision: 0.9495306036922946
best accuracy model recall: 0.9496040316774659
best accuracy model f1: 0.9495154521287027
```

final summary table

The same random forest model performed best in both tasks, but with different thresholds. The default threshold of 0.5 was best for overall predictive performance, while the stricter threshold of 0.75 was best for minimising cost under the 10:1 penalty structure. This shows that the best decision rule depends on what you care about most.

```
# final metrics for default rf model

rf_default_fp = rf_100_10_preds.filter(
  (F.col("spam") == 0) & (F.col("prediction") == 1)
).count()

rf_default_fn = rf_100_10_preds.filter(
  (F.col("spam") == 1) & (F.col("prediction") == 0)
).count()

rf_default_total = rf_100_10_preds.count()

rf_default_avg_cost = (10 * rf_default_fp + rf_default_fn) / rf_default_total

precision_eval = MulticlassClassificationEvaluator(
  labelCol="spam",
  predictionCol="prediction",
  metricName="weightedPrecision"
)

recall_eval = MulticlassClassificationEvaluator(
  labelCol="spam",
  predictionCol="prediction",
  metricName="weightedRecall"
)

f1_eval = MulticlassClassificationEvaluator(
```

```
  labelCol="spam",
  predictionCol="prediction",
  metricName="f1"
)

rf_default_precision = precision_eval.evaluate(rf_100_10_preds)
rf_default_recall = recall_eval.evaluate(rf_100_10_preds)
rf_default_f1 = f1_eval.evaluate(rf_100_10_preds)

# final metrics for cost-sensitive rf model

cost_precision_eval = MulticlassClassificationEvaluator(
  labelCol="spam",
  predictionCol="custom_prediction",
  metricName="weightedPrecision"
)

cost_recall_eval = MulticlassClassificationEvaluator(
  labelCol="spam",
  predictionCol="custom_prediction",
  metricName="weightedRecall"
)

cost_f1_eval = MulticlassClassificationEvaluator(
  labelCol="spam",
  predictionCol="custom_prediction",
  metricName="f1"
)

rf_cost_precision = cost_precision_eval.evaluate(rf_cost_preds)
rf_cost_recall = cost_recall_eval.evaluate(rf_cost_preds)
rf_cost_f1 = cost_f1_eval.evaluate(rf_cost_preds)

#final summary table


final_summary_rows = [
  (
    "rf depth 10 threshold 0.5",
    rf_100_10_accuracy,
    rf_100_10_auc,
    rf_default_avg_cost,
    rf_default_precision,
    rf_default_recall,
    rf_default_f1
  ),
```

```
  (
    "rf depth 10 threshold 0.75",
    rf_cost_accuracy,
    rf_100_10_auc,
    rf_avg_cost,
    rf_cost_precision,
    rf_cost_recall,
    rf_cost_f1
  )
]

final_summary_df = spark.createDataFrame(
  final_summary_rows,
  ["model", "accuracy", "auc", "avg_cost", "precision", "recall", "f1"]
)

final_summary_df.show(truncate=False)
```

```
+-------------------------+------------------+------------------+-------------------+------------------+------------------+-----------
|model                    |accuracy          |auc               |avg_cost           |precision         |recall            |f1
+-------------------------+------------------+------------------+-------------------+------------------+------------------+-----------
|rf depth 10 threshold 0.5 |0.9496040316774658|0.9858758048674009|0.24478041756659466|0.9495306036922946|0.9496040316774659|0.949515452
|rf depth 10 threshold 0.75|0.9208063354931606|0.9858758048674009|0.1375089992800576 |0.9265743483820572|0.9208063354931606|0.919144575
+-------------------------+------------------+------------------+-------------------+------------------+------------------+-----------
```

```
print("best accuracy model confusion matrix")
rf_100_10_preds.groupBy("spam", "prediction") \
    .count() \
    .orderBy("spam", "prediction") \
    .show()

print("best cost-sensitive model confusion matrix")
rf_cost_preds.groupBy("spam", "custom_prediction") \
    .count() \
    .orderBy("spam", "custom_prediction") \
    .show()
```

```
best accuracy model confusion matrix
+----+----------+-----+
|spam|prediction|count|
+----+----------+-----+
| 0.0|       0.0|  820|
| 0.0|       1.0|   30|
| 1.0|       0.0|   40|
| 1.0|       1.0|  499|
```

```
+----+----------+-----+

best cost-sensitive model confusion matrix
+----+-----------------+-----+
|spam|custom_prediction|count|
+----+-----------------+-----+
| 0.0|              0.0|  841|
| 0.0|              1.0|    9|
| 1.0|              0.0|  101|
| 1.0|              1.0|  438|
+----+-----------------+-----+
```

The default threshold catches more spam overall, which improves recall and accuracy, but it also creates more false positives. Raising the threshold to 0.75 makes the classifier more conservative, which lowers false positives and therefore lowers average cost, even though some spam is missed.

The random forest with 100 trees and depth 10 gave the best overall accuracy. Using the default threshold of 0.5, it reached 94.96% accuracy and an AUC of 0.9859.